

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# -----
# Load image (create one if not found)
# -----
image_path = "/content/aishu photo (2).avif"

if not os.path.exists(image_path):
    # Create a synthetic grayscale image
    img = np.zeros((256, 256), dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (200, 200), 150, -1)
    cv2.circle(img, (128, 128), 40, 220, -1)
else:
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# -----
# Add salt-and-pepper noise
# -----
noisy = img.copy()
p = 0.02 # noise probability
rnd = np.random.rand(*img.shape)

noisy[rnd < p / 2] = 0 # pepper noise
noisy[rnd > 1 - p / 2] = 255 # salt noise

# -----
# Gaussian smoothing
# -----
gaussian = cv2.GaussianBlur(noisy, (5, 5), 1.0)

# -----
# Median filtering
# -----
median = cv2.medianBlur(noisy, 5)

# -----
# Display results
# -----
titles = ["Original Image", "Noisy Image", "Gaussian Smoothing", "Median Filtering"]
images = [img, noisy, gaussian, median]

plt.figure(figsize=(10, 6))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

Original Image



Noisy Image



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image_path = "/content/aishu photo (2).avif" # change path if needed
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Check if image loaded correctly
if img is None:
    raise FileNotFoundError(f"Could not load image at path: {image_path}")

# ----- 1) Histogram Equalization -----
hist_eq = cv2.equalizeHist(img)

# ----- 2) Contrast Stretching (Min-Max Normalization) -----
min_val = np.min(img)
max_val = np.max(img)

# Prevent division by zero
if max_val > min_val:
    contrast_stretched = ((img - min_val) * (255.0 / (max_val - min_val))).astype(np.uint8)
else:
    contrast_stretched = img.copy()

# ----- Display Results -----
titles = ["Original Image", "Histogram Equalization", "Contrast Stretching"]
images = [img, hist_eq, contrast_stretched]

plt.figure(figsize=(12, 4))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Histogram Equalization



Contrast Stretching



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image_path = "/content/aishu photo (2).avif" # Change path if needed
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Check if image loaded properly
if img is None:
```

```

raise FileNotFoundError(f"Could not load image at path: {image_path}")

# ----- Preprocessing -----
# Slight blur to reduce noise before edge detection
blur = cv2.GaussianBlur(img, (3, 3), 0)

# ----- 1) Sobel Edge Detection -----
sobelx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)

# Gradient magnitude
sobel_mag = cv2.magnitude(sobelx, sobely)
sobel_mag = cv2.convertScaleAbs(sobel_mag)

# ----- 2) Canny Edge Detection -----
canny_edges = cv2.Canny(blur, threshold1=100, threshold2=200)

# ----- Display Results -----
titles = ["Original Image", "Sobel Magnitude", "Canny Edges"]
images = [img, sobel_mag, canny_edges]

plt.figure(figsize=(12, 4))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

Original Image



Sobel Magnitude



Canny Edges



```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image_path = "/content/aishu photo (2).avif" # Change path if needed
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Check if image loaded correctly
if img is None:
    raise FileNotFoundError(f"Could not load image at path: {image_path}")

# ----- Create Blurred + Noisy Image -----
blurred = cv2.GaussianBlur(img, (7, 7), 1.5)

# Add Gaussian noise safely
noise = np.random.normal(0, 10, img.shape).astype(np.float32)
noisy_blurred = blurred.astype(np.float32) + noise
noisy_blurred = np.clip(noisy_blurred, 0, 255).astype(np.uint8)

# ----- 1) Deblurring (Sharpening Filter) -----
sharpen_kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]], dtype=np.float32)

deblurred = cv2.filter2D(noisy_blurred, -1, sharpen_kernel)

# ----- 2) Denoising (Non-Local Means) -----
denoised = cv2.fastNlMeansDenoising(
    noisy_blurred,
    None,

```

```

h=15,
templateWindowSize=7,
searchWindowSize=21
)

# ----- Display Results -----
titles = [
    "Original Image",
    "Noisy + Blurred",
    "Deblurred (Sharpening)",
    "Denoised (NLM)"
]

images = [img, noisy_blurred, deblurred, denoised]

plt.figure(figsize=(10, 8))
for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

Original Image



Noisy + Blurred



Deblurred (Sharpening)



Denoised (NLM)



```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- Load Image -----
image_path = "/murugaa.jpeg" # Change path if needed
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Check if image loaded correctly
if img is None:
    raise FileNotFoundError(f"Could not load image at path: {image_path}")

```

```
# ----- Binary Conversion (Otsu Thresholding) -----
_, binary = cv2.threshold(
    img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# ----- Structuring Element -----
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# ----- Morphological Operations -----
eroded = cv2.erode(binary, kernel, iterations=1)
dilated = cv2.dilate(binary, kernel, iterations=1)
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

# ----- Display Results -----
titles = [
    "Original Grayscale",
]

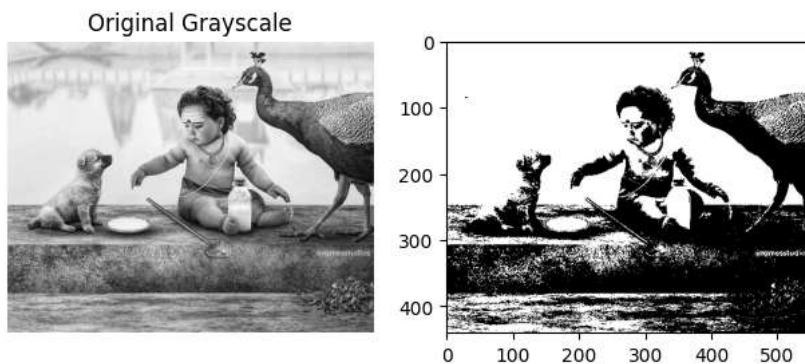
images = [img, binary, eroded, dilated, opened, closed]

plt.figure(figsize=(12, 8))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.show()
```

```
-----
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-1344459308.py in <cell line: 0>()
    37     plt.subplot(2, 3, i + 1)
    38     plt.imshow(images[i], cmap="gray")
--> 39     plt.title(titles[i])
    40     plt.axis("off")
    41
```

IndexError: list index out of range



```
from google.colab import drive
drive.mount('/content/drive')
```

